

Security Doorbell Cam

Chain K — Hardware & Systems Foundations · Full written lesson

A commercial smart doorbell is a camera, a PIR sensor, a speaker/mic, and a subscription that ships your footage to someone else's server. Building the same thing means the footage stays on hardware you control.

Lesson 1 — Interfacing the Camera and PIR, and Debouncing False Triggers

A commercial doorbell hides two sensors behind one button-shaped device: a camera and a motion sensor, each with its own real quirks worth understanding before writing a line of detection code.

- The Pi Camera connects over CSI and captures frames or snapshots through a straightforward API; a PIR sensor connects to a single GPIO pin and outputs a simple HIGH/LOW signal.
- The naive approach — notify on every HIGH reading — fires constantly. Sunlight, heat sources, and moving foliage all produce brief HIGH flickers that aren't a person at the door.
- The real fix is debouncing: require the signal to stay HIGH for a minimum sustained duration before confirming a real event, and add a cooldown period after each confirmed event.

The trap: PIR false-triggers from sunlight, heat sources, or moving foliage. A PIR sensor with no adjustable sensitivity/delay pots makes this worse — tune both the hardware pots and a software debounce window.

Worked example: a raw PIR signal with 6 separate HIGH flickers reduces to exactly 2 confirmed events once a minimum-duration debounce and cooldown are applied.

Lesson 2 — Budgeting a Detection Pipeline's Real Latency

This project is a real, complete edge-computing pipeline in miniature — and its usefulness depends on how fast it runs end to end, not just whether it eventually works.

- The pipeline's shape: sensor trigger → local inference/decision → notification.
- A motion alert that arrives 30 seconds after someone's already left isn't useful — this project's own target is becoming a phone alert in under a couple of seconds.
- Each stage has a real time cost: capturing a frame, running the decision logic, and the network round-trip to a push service.

The trap: notification latency when the detection pipeline is too heavy for a Zero-class board. Running a full object-detection model on hardware that's not fast enough blows the latency budget regardless of the algorithm.

Worked example: a pipeline budgeting capture at 150ms, inference at 800ms, and notify at 400ms totals 1,350ms — under a 2,000ms target. Swap in a heavier inference stage at 2,000ms alone and the pipeline blows the budget even with the other stages unchanged.

Lesson 3 — Wiring a Real Intercom Without Feedback

Two-way audio is what turns a camera-plus-notification setup into an actual intercom.

- A microphone captures the visitor's voice; a speaker (driven through an amplifier) plays the resident's voice back.
- The real risk in a small enclosure is acoustic feedback: sound from the speaker travels back into the microphone, gets amplified again, and squeals.
- The fix is real physical separation between the speaker and mic within the enclosure.

The trap: running the amp/speaker too close to the mic, causing feedback in a small enclosure — the same acoustic problem Walkie-Talkie-Build's audio hardware faces.

Worked example: the same speaker-and-mic hardware, mounted close together, produces audible feedback within seconds; repositioned with real physical separation, it runs cleanly with no code changes.

Lesson 4 — Local Footage Retention, and Weatherproofing Camera vs. PIR Differently

Keeping footage on your own hardware means actually managing that storage — and, if mounted outside, protecting two different sensors in two genuinely different ways.

- Continuous recording fills an SD card fast and wears it out faster; motion-triggered recording uses far less storage and far fewer writes.
- A real rolling-storage policy (a max total size or max age) is the actual fix for "the SD card filled up" — the same pattern real NVR/surveillance software uses.
- The camera can sit behind a sealed clear acrylic window; the PIR sensor's fresnel lens has to stay physically exposed since it needs direct IR line-of-sight.

The trap: letting local footage fill the SD card with no retention policy at all.

Worked example: the same enclosure design uses two different physical solutions for two different sensing problems — sealed acrylic for the camera, a bare gasket-ringed opening for the PIR.

Reflection

What's the actual tradeoff between debouncing aggressively (fewer false alerts) and missing brief real motion events? Why does this project count as "a real edge-computing pipeline in miniature," and what would change if you scaled it to 50 doorbells instead of 1?